

Contents

Criterion Functions in Clustering	1
Sum of Squared Error Criterion (SSE)	1
Minimum Variance Criterion	1
Scatter Criteria	2
K-Means Clustering	2
Hierarchical Clustering	4
DBSCAN Algorithm (Density-Based Clustering)	8

Criterion Functions in Clustering

Criterion functions (also called objective functions or cost functions) are mathematical functions used to evaluate the quality of a clustering solution. They guide the clustering process by providing a way to measure how well the data points have been grouped.

Why are Criterion Functions Needed?

- To **quantify** how well a clustering solution fits the data.
- To **compare** different clustering results objectively.
- To **optimize** cluster formation by minimizing or maximizing a specific mathematical function.
- To make clustering **unsupervised learning** more structured and goal-driven.

Sum of Squared Error Criterion (SSE)

- Also known as **Intra-Cluster Variance** or **Within-Cluster Sum of Squares (WCSS)**.
- It is the **sum of squared distances** between each point and its **own cluster center**.
- **Objective: Minimize** SSE for better clustering (i.e., tighter, more compact clusters).

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- k = number of clusters
- C_i = points in cluster i
- μ_i = centroid of cluster i

Minimum Variance Criterion

This is **directly related** to SSE and often used in **K-Means** clustering.

Relation:

- It is the **same as minimizing the SSE**.
- Thus, **minimum variance criterion** is a **clustering objective** that seeks to:
 - Minimize the **intra-cluster variance** (i.e., make clusters as compact as possible).

Key Point:

- Often used synonymously with SSE in clustering context.
- Variance is just SSE **divided by number of points**.

Scatter Criteria

Scatter matrices are used to **analyze cluster structure** and are mostly applied in **pattern recognition** and **clustering evaluation**.

There are **three types**:

a. Within-Cluster Scatter Matrix (S_W):

- Measures how spread out the data is **within** each cluster.
- Should be **minimized** for better clustering.

$$S_W = \sum_{i=1}^k \sum_{x \in C_i} (x - \mu_i)(x - \mu_i)^T$$

b. Between-Cluster Scatter Matrix (S_B):

- Measures how far the **cluster centers** are from the **global mean**.
- Should be **maximized** to ensure well-separated clusters.

$$S_B = \sum_{i=1}^k n_i (\mu_i - \mu)(\mu_i - \mu)^T$$

c. Total Scatter Matrix (S_T):

- Represents total variability in the dataset.
- Helps decompose total variability into **within** and **between** components.

$$S_T = S_W + S_B$$

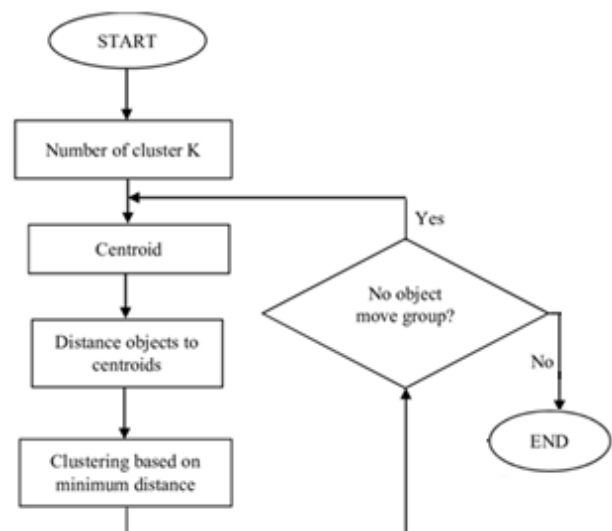
K-Means Clustering

- K-Means is a **partition-based** unsupervised clustering algorithm that divides a dataset into **K non-overlapping clusters**, where each data point belongs to the cluster with the **nearest mean** (centroid).
- Objective:** Minimize the within-cluster sum of squared distances (or variance) between data points and their assigned cluster centroids.
- K:** The number of clusters to be formed, which must be specified in advance
- Cluster:** A group of similar data points.
- Centroid:** The center (mean) of a cluster.
- Distance metric:** typically, Euclidian distance

Objective Function (Loss function):

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- k = number of clusters
- C_i = points in cluster i
- μ_i = centroid of cluster i



Advantages:

- Simple and fast to implement.
- Efficient for large datasets.
- Works well with **spherical, equally sized** clusters.

Disadvantages:

- Requires pre-specifying the number of clusters k .
- Sensitive to **initial centroid positions**.
- Not suitable for **non-convex** clusters.
- Struggles with **outliers** and **noisy data**.

Applications:

- Image segmentation
- Market/customer segmentation
- Document clustering
- Anomaly detection

Example

- For $K=2$ perform K means clustering algorithm. Assume A and C are the initial centroids.
- Calculate objective/Loss function of the centroid after they converge.

Point	X	Y
A	2	10
B	2	5
C	8	4
D	5	8
E	7	5
F	6	4

Centroids: A (2,10) & C (8,4)

Point	Distance from A	Distance from C	Ans
A(2,10)	0	8.4853	C_1
B(2,5)	5	6.0828	C_1
C(8,4)	8.4853	0	C_2
D(5,8)	3.6056	5	C_1
E(7,5)	7.0711	1.4142	C_2
F(6,4)	8.9442	2	C_2

$\therefore$ Cluster 1: A, B, D

Cluster 2: C, E, F

$$\text{New centroid} = \left(\frac{2+2+5}{3}, \frac{10+5+8}{3} \right) = (3, 7.6667)$$

(~~C~~) (C_1)

$$\text{New centroid} = \left(\frac{8+7+6}{3}, \frac{4+5+4}{3} \right) = (7, 4.3333)$$

(C_2)

Point	Dis from c_1	Dis from c_2	Ans.
A(2,10)	2.5383	5.5378	c_1
B(2,3)	2.8480	5.0443	c_1
C(8,4)	6.2004	1.0541	c_2
D(3,8)	2.0278	4.1767	c_1
E(7,5)	4.8074	0.6667	c_2
F(6,4)	4.7376	1.0541	c_2

Same c_1 & c_2 for both iteration

$\therefore$ Convergence.

$$\text{Loss function}(c_1) = \sum (x_i - c_1)^2$$

$$= (2.5383)^2 + (2.8480)^2 + (2.0278)^2 = 18.6652$$

$$\text{Loss func}(c_2) = (1.0541)^2 + (0.6667)^2 + (1.0541)^2 = 2.6667$$

$$\text{Total Loss function} = 18.6652 + 2.6667 = 21.3319$$

Hierarchical Clustering

- Hierarchical Clustering is an **unsupervised learning algorithm** that builds a hierarchy of clusters either from the **bottom-up** (agglomerative) or **top-down** (divisive).
- It does not require the number of clusters to be defined in advance.
- It groups data points together that are close to each other based on the measure of distance.

Type	Description
Agglomerative (Bottom-Up)	Start with each data point as its own cluster. Iteratively merge the closest clusters until one cluster remains or a stopping criterion is met.
Divisive (Top-Down)	Start with all data in one cluster. Iteratively split clusters until each data point is in its own cluster or a stopping criterion is met.

Dendrogram:

- A **tree-like diagram** used to represent the sequence of merges or splits.
- The **height** of each node in the dendrogram indicates the **distance** (or dissimilarity) at which clusters are joined or divided.
- To **decide the number of clusters**, draw a **horizontal cut** at the desired height.

Linkage Criteria (to compute inter-cluster distance):

Linkage Type	Description
--------------	-------------

Single Linkage	Minimum distance between any two points (nearest neighbor)
Complete Linkage	Maximum distance between any two points (farthest neighbor)
Average Linkage	Average distance between all pairs of points from two clusters
Centroid Linkage	Distance between centroids of clusters
Ward's Method	Minimizes the total within-cluster variance

Advantages:

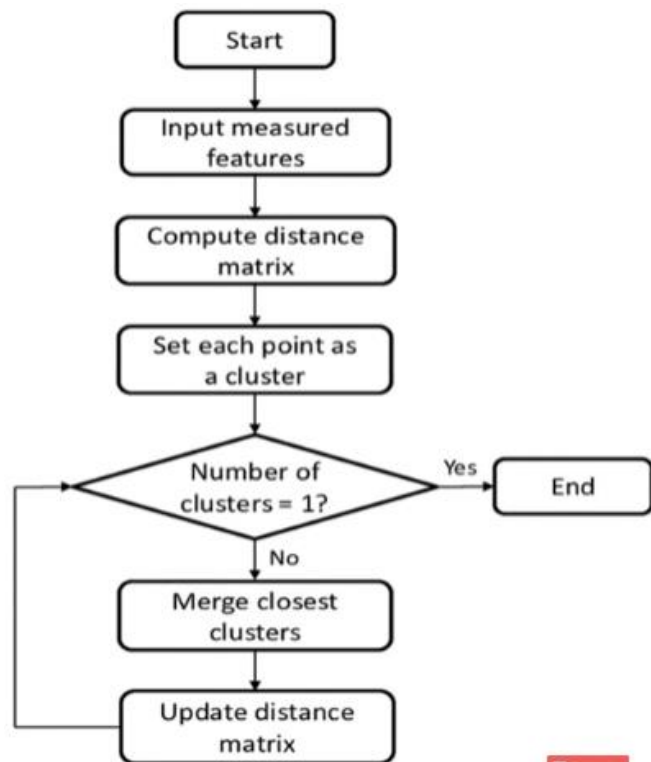
- No need to specify number of clusters in advance.
- Can produce meaningful hierarchy and dendrogram visualization.
- Flexible linkage methods.

Disadvantages:

- **Computationally expensive** for large datasets (time complexity: $O(n^3)$ and space $O(n^2)$).
- Once a merge or split is done, it **cannot be undone**.
- Sensitive to noise and outliers.
- Dendrogram can become difficult to interpret for large data.

Applications:

- Gene expression analysis
- Taxonomy and biology (organism classification)
- Social network analysis
- Document or web page clustering



Single Linkage(min) Agglomerative Clustering

	18	22	25	27	42	43
18	0					
22	4	0				
25	7	3	0			
27	9	5	2	0		
42	24	20	17	15	0	
43	25	21	18	16	1	0

Min distance between pair of points is 42 & 43.
So we can merge them.

We consider Single Linkage (min distance)

	18	22	25	27	(42, 43)
18	0				
22	4	0			
25	7	3	0		
27	9	5	2	0	
(42, 43)	24	20	17	15	0

Min distance is (25, 27)

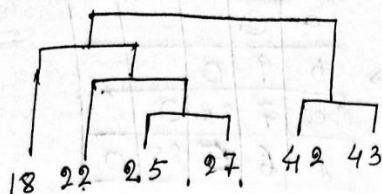
	18	22	(25, 27)	(42, 43)
18	0			
22	4	0		
(25, 27)	7	3	0	
(42, 43)	24	20	15	0

	18	(22, 25, 27)	(42, 43)
18	0		
(22, 25, 27)	4	0	
(42, 43)	24	15	0

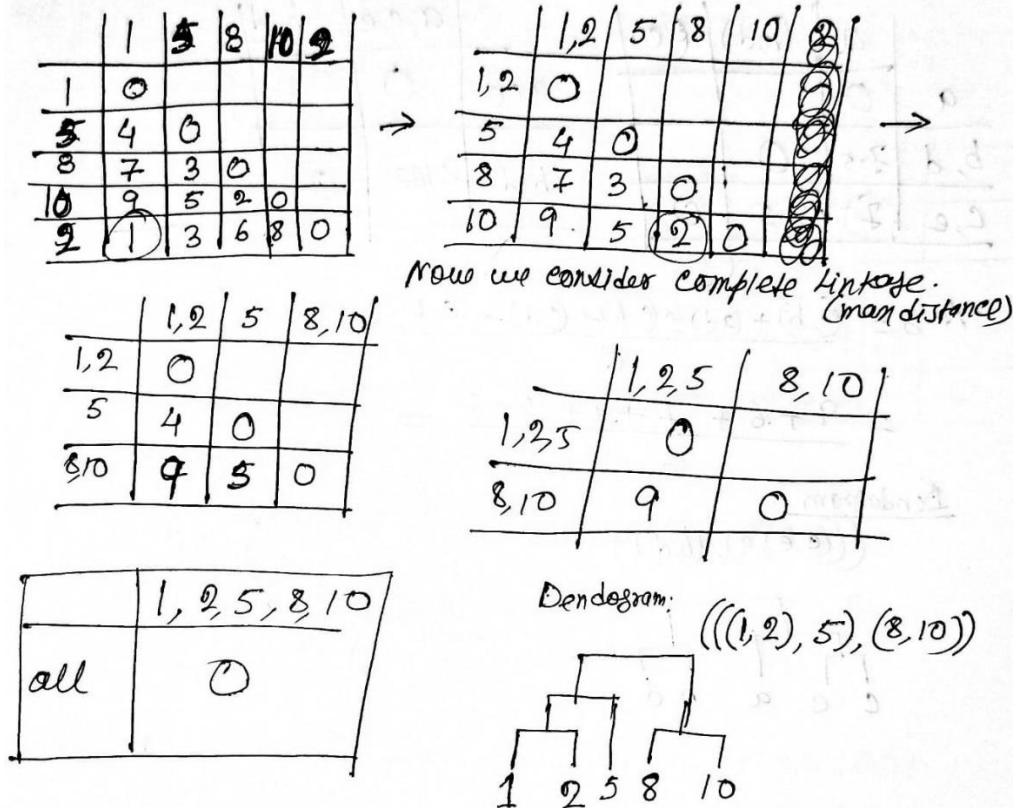
	(18, 22, 25, 27)	(42, 43)
(18, 22, 25, 27)	0	
(42, 43)	15	0

	all
all	0

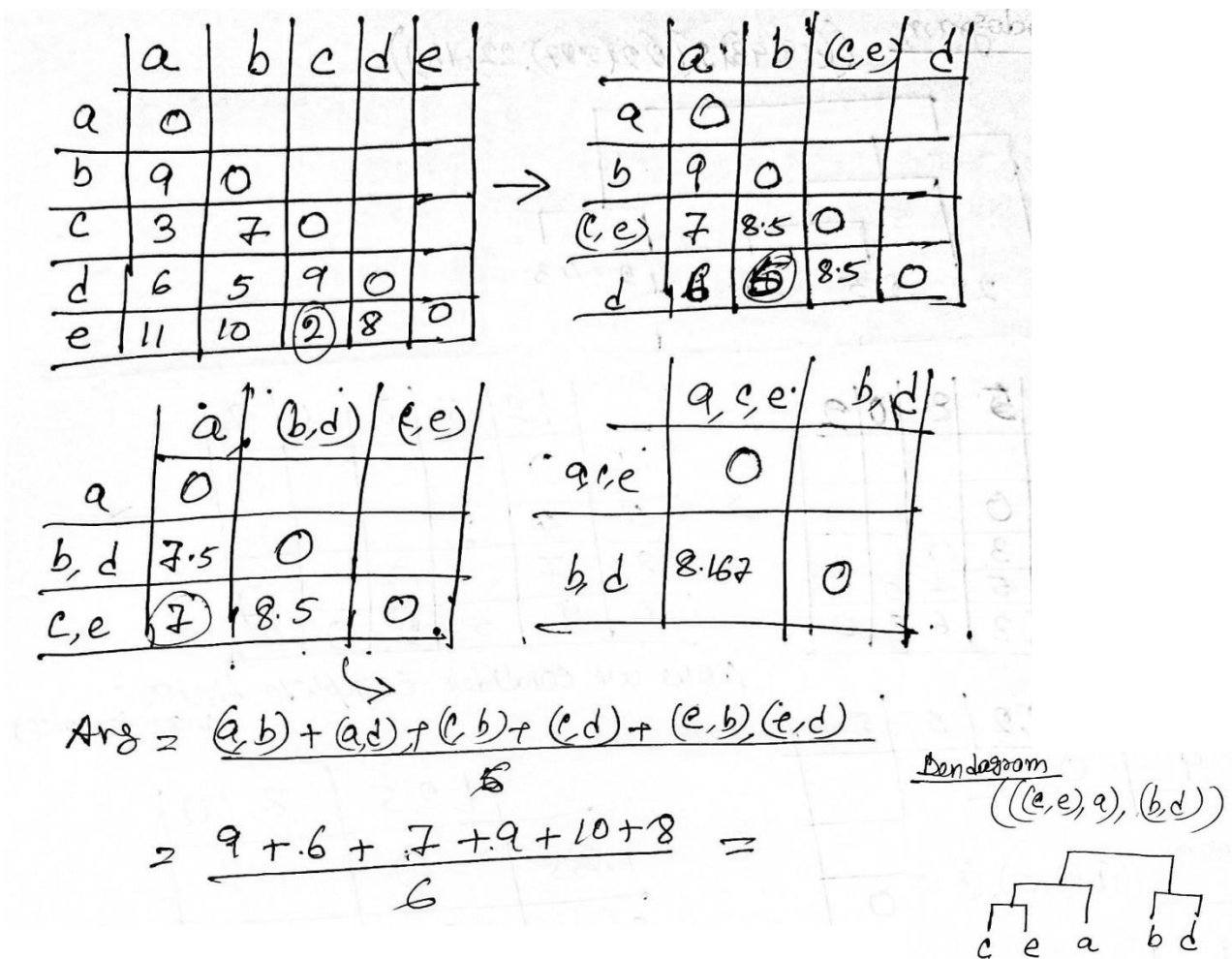
Dendrogram ((42, 43), ((25, 27), 22), 18))



Complete Linkage (Max) Agglomerative Clustering



Average Linkage Agglomerative Clustering



DBSCAN Algorithm (Density-Based Clustering)

DBSCAN is a **density-based clustering algorithm** that groups together points that are **closely packed** (high-density regions) and marks **isolated points** (low-density) as **noise/outliers**.

Unlike K-Means, **no need to predefine number of clusters**.

1. **ϵ (Epsilon):** Radius around a point to search for neighboring points.
2. **MinPts (Minimum Points):** Minimum number of points required within ϵ -neighborhood to form a **dense region**.
3. **Core Point:** A point with at least **MinPts** neighbors (including itself) within ϵ radius.
4. **Border Point:** A point that lies within the ϵ radius of a core point but **has fewer than MinPts** neighbors.
5. **Noise (Outlier):** A point that is **neither a core point nor a border point**.

Advantages:

- Can **detect clusters of arbitrary shape** (unlike K-Means).
- **No need to specify number of clusters** beforehand.
- **Identifies outliers** (noise).
- Works well with **spatial data**.

Disadvantages:

- Struggles with **varying densities** (ϵ and MinPts are global).
- **Parameter selection** (ϵ , MinPts) is not trivial.
- Not efficient for **high-dimensional data**.

Applications:

- Geographical mapping (e.g., earthquakes, crime hot-spots)
- Image segmentation
- Anomaly detection
- Market segmentation

	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆	P ₇	P ₈	P ₉	P ₁₀	P ₁₁	P ₁₂
P ₁	0											
P ₂	1.41	0										
P ₃	2.83	1.41	0									
P ₄	4.24	2.83	1.41	0								
P ₅	5.66	4.24	2.83	1.41	0							
P ₆	5.83	4.42	3.16	2.0	1.41	0						
P ₇	6.40	5.00	3.61	2.24	2.00	1.0	0					
P ₈	5.83	4.42	3.16	2.00	1.41	2.83	2.24	0				
P ₉	4.00	3.16	2.83	2.16	4.00	3.16	4.12	5.10	0			
P ₁₀	1.41	2.00	3.16	4.42	5.83	5.66	6.4	6.32	3.16	0		
P ₁₁	2.00	1.41	2.00	3.16	4.42	4.24	5.0	5.1	2.0	1.41	0	
P ₁₂	3.16	2.83	3.16	4.00	5.1	4.42	5.39	6.0	1.41	2.0	1.41	0

minpts = 4, $E = 1.9$

P₁: P₂, P₁₀ P₄: P₃, P₅ P₇: P₅, P₆ P₁₀: P₁, P₁₁
 P₂: P₁, P₃, P₁₁ P₅: P₄, P₆, P₇, P₈ P₈: P₅ P₁₁: P₂, P₁₀, P₁₂
 P₃: P₂, P₄ P₆: P₅, P₇ P₉: P₁₂ P₁₂: P₉, P₁₂

Point	Status	
P ₁	noise	border
P ₂	core	—
P ₃	noise	border
P ₄	noise	border
P ₅	core	—
P ₆	noise	border
P ₇	noise	border
P ₈	noise	border
P ₉	noise	
P ₁₀	noise	border
P ₁₁	core	—
P ₁₂	noise	border

P₁: P₂, P₁₀ means total data points are 3.

minpts = 4

if < 4 → noise
else → core

